

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
дисциплины
«Искусственный интеллект и машинное обучение»

Выполнил:
Червиченко Иван Денисович
2 курс, группа ИВТ-б-о-24-2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института
перспективной инженерии Воронкин
Р.А

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Цель работы: исследовать базовые возможности интерактивных оболочек Jupyter Notebook, JupyterLab и Google Colab для языка программирования Python.

Ссылка на репозиторий: <https://github.com/progwar/ailab1>

Ход работы:

1. Был создан общедоступный репозиторий, в котором использована лицензия MIT и язык программирование Python.

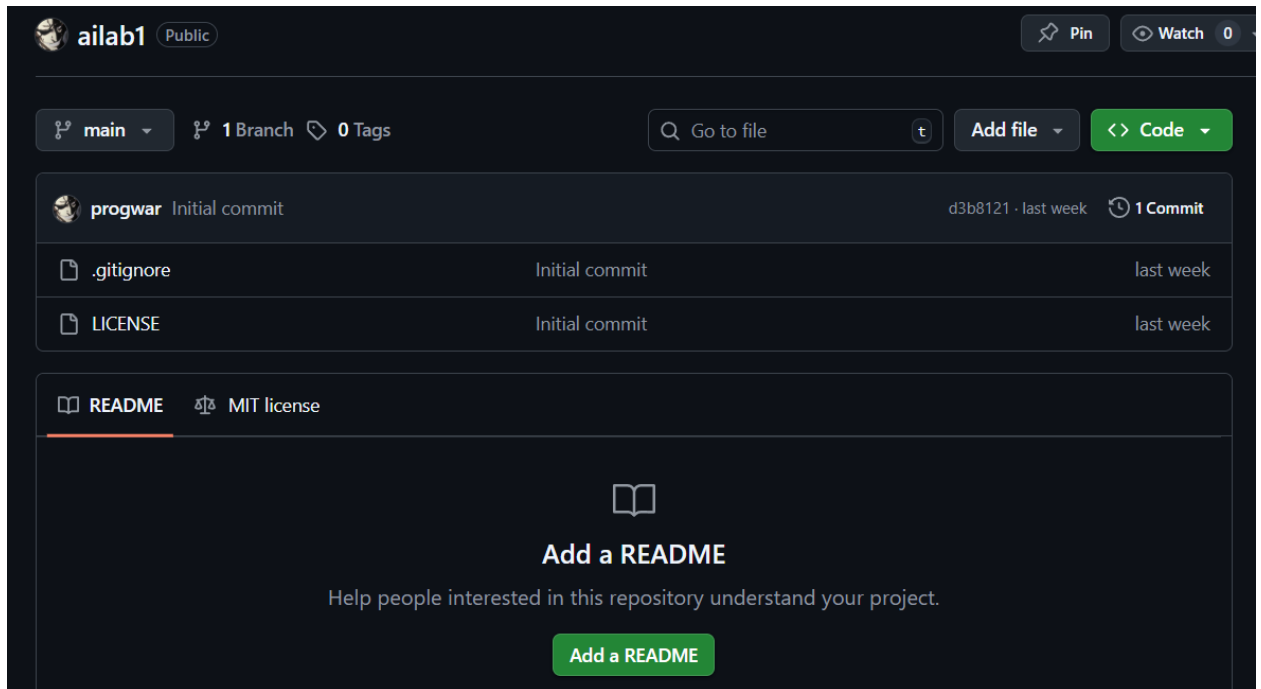


Рисунок 1. Созданный репозиторий

2. Репозиторий был клонирован на рабочий компьютер.

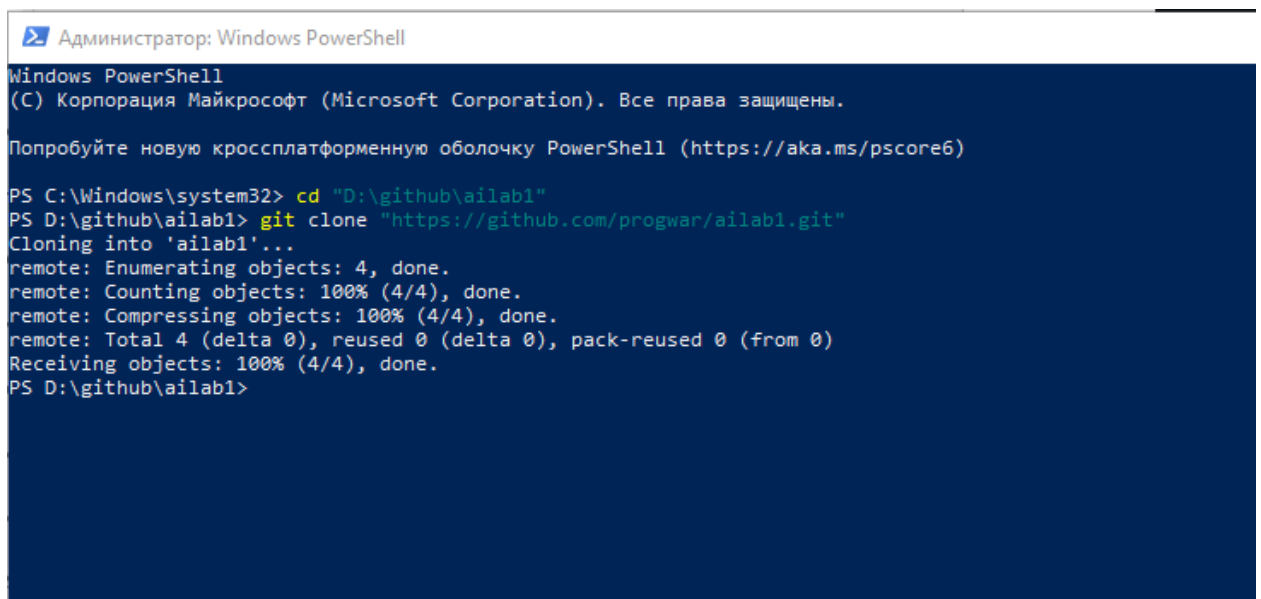


Рисунок 2. Клонирование репозитория

3. Файл .gitignore был дополнен необходимыми правилами.

A screenshot of a text editor window titled ".gitignore - Блокнот". The window has a menu bar with "Файл", "Правка", "Формат", "Вид", and "Справка". The main text area contains a .gitignore file with the following content:

```
# Flask stuff:
instance/
.webassets-cache

# Scrapy stuff:
.scrapy

# Sphinx documentation
docs/_build/

# PyBuilder
.pybuilder/
target/

# Jupyter Notebook
.ipynb_checkpoints/
*.ipynb_checkpoints*
*/.ipynb_checkpoints/*
.ipynb_checkpoints/*
*.ipynb.json
*/.ipynb_checkpoints/
~*.ipynb
*.ipynb.py
.ipynb_checkpoints/
*.pkl
*.pickle

# JupyterLab
.jupyter/
jupyter_config.json
jupyter_notebook_config.json
jupyter_notebook_config.py
jupyterlab-workspace.json

# IPython
profile_default/
ipython_config.py

# pyenv
# For a library or package, you might want to ignore these files since the code is
# intended to run in multiple environments; otherwise, check them in:
# .python-version

# pipenv
# According to pya/pipenv#598, it is recommended to include Pipfile.lock in version control.
# However, in case of collaboration, if having platform-specific dependencies or dependencies
# having no cross-platform support, pipenv may install dependencies that don't work, or not
# install all needed dependencies.
#Pipfile.lock
```

Рисунок 3. Дополненный .gitignore файл

4. Были проработаны примеры лабораторной работы.

```
[2]: 3+2
```

5

```
[4]: a = 5
      b = 7
      print(a + b)
```

12

```
[6]: n = 7
      for i in range(n):
          print(i*10)
```

0
10
20
30
40
50
60

```
[9]: i = 0
      while True:
          i += 1
          if i > 5:
              break
          print("Test while")
```

Test while
Test while
Test while
Test while
Test while

Рисунок 4. Пример 1

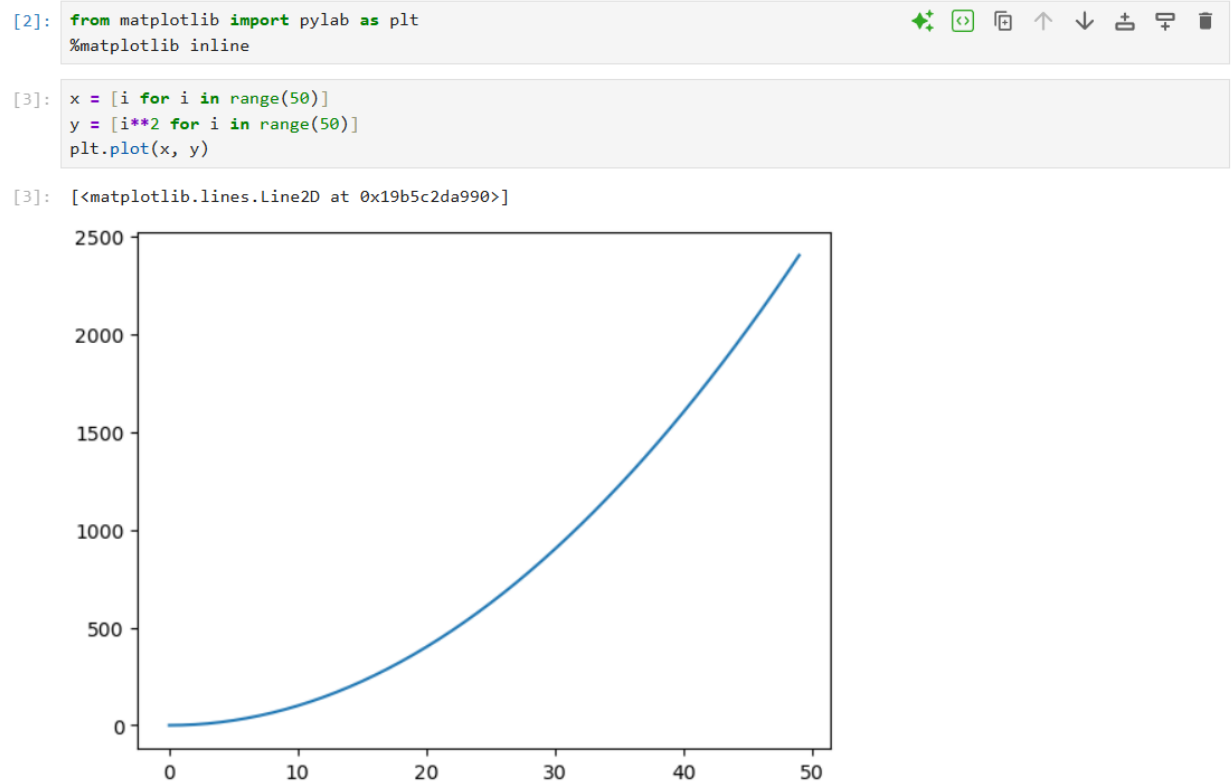


Рисунок 5. Пример 2

```
[2]: %%time
import time
for i in range(50):
    time.sleep(0.1)

CPU times: total: 15.6 ms
Wall time: 5.07 s

[3]: %timeit x = [(i**10) for i in range(10)]

1.88 µs ± 57.6 ns per loop (mean ± std. dev. of 7 runs, 1,000,000 loops each)
```

Рисунок 6. Пример 3

▼ Заголовок первого уровня

Заголовок второго уровня

Полужирный текст, *курсив*, код в строке

Список:

- Пункт 1
- Пункт 2
- Пункт 3

Формула: $y = mx + b$

Рисунок 7. Пример 4

```
[1]: %timeit sum(range(1000))

19.1 µs ± 724 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

Рисунок 8. Пример 5

```
[1]: %%time
total = 0
for i in range(10**6):
    total += i

CPU times: total: 266 ms
Wall time: 265 ms
```

Рисунок 9. Пример 6

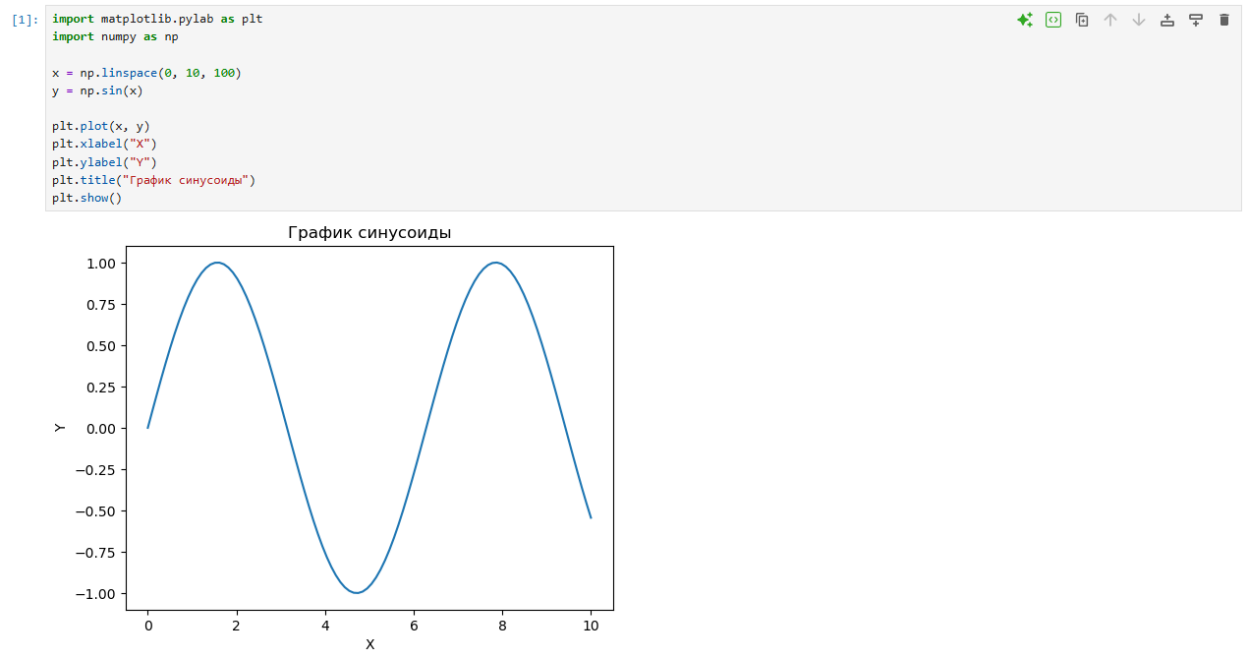


Рисунок 10. Пример 7

5. Были проработаны задания практической работы.

Задания практической работы

Работа с ячейками и Markdown

Цель: освоить работу с различными типами ячеек (код, Markdown) и форматированием текста.

Задания:

Создайте новую **Markdown-ячейку** и:

- Напишите заголовок "Практическое задание %1" .
- Добавьте **жирный** и *курсивный* текст.
- Создайте нумерованный и маркированный списки.
- Вставьте согласно индивидуального задания, например:
- Вставьте изображение через `![Описание](URL)` .

Создайте ячейку **Python-кода** и:

- Запросите у пользователя его имя с помощью `input()` .
- Выведите приветствие: "Привет, <имя>! Добро пожаловать в JupyterLab / Google Colab!" .
- Запустите ячейку (`Shift + Enter`).

Рисунок 10. Задание 1

Практическое задание №1

Жирный текст и курсив

Нумерованный список:

1. Первое
2. Второе
3. Третье

Маркированный список:

- Первое
- Второе
- Третье

Преобразование Лапласа функций:

$$F(p) = \int_0^{\infty} e^{-pt} f(t) dt$$

Изображение:



```
name = input("Введите имя")  
print(f"Привет, {name}! Добро пожаловать в JupyterLab!")
```

Введите имя Имя
Привет, Имя! Добро пожаловать в JupyterLab!

Рисунок 11. Выполненное задание 1 (по варианту)

Работа с файлами

Цель: изучить загрузку, создание и сохранение файлов в JupyterLab / Google Colab.

Задания:

1. Создайте и сохраните текстовый файл с помощью `open()` .
2. Запишите в него несколько строк текста.
3. Закройте файл и затем откройте его снова, считав содержимое и выведя на экран.
4. Проверьте, существует ли файл, используя `os.path.exists()` .
5. Удалите файл с помощью модуля `os` .

Рисунок 12. Задание 2

```
[10]: import os

with open("file.txt", "w") as f:
    f.write("Строка\n")
    f.write(" Строка\n")
    f.write(" Строка\n")
with open("file.txt", "r") as f:
    content = f.read()
    print("Содержимое файла:\n", content)

print("Файл существует", os.path.exists("file.txt"))

Содержимое файла:
Строка
Строка
Строка

Файл существует

[12]: os.remove("file.txt")
print("Файл удалён.")

Файл удалён.
```

Рисунок 13. Выполненное задание 2

Магические команды Jupyter

Цель: изучить магические команды для удобной работы в JupyterLab / Google Colab.

Задания:

1. Выведите список всех доступных магических команд (`%lsmagic`).
2. Используйте `%time` и `%%timeit` для измерения времени выполнения кода.
3. Создайте Python-скрипт в Jupyter (`%%writefile script.py`) и выполните его через `!python script.py`.
4. Выведите список файлов в текущей директории с помощью `%ls`.
5. Используйте `%history` для просмотра истории команд.

Рисунок 14. Задание 3


```
[2]: %lsmagic
root
cell
line

[4]: %time
CPU times: total: 0 ns
Wall time: 7.15 µs

[9]: %%writefile script.py
print("x y z w")
for x in range(0, 2):
    for y in range(0, 2):
        for z in range(0, 2):
            for w in range(0, 2):
                if not(((x <= y) and (y <= w)) or (z == (x or y))):
                    print(x, y, z, w)

Overwriting script.py

[17]: !python script.py
x y z w
0 1 0 0
1 0 0 0
1 0 0 1
1 1 0 0

[13]: %time sum(range(1000000))
CPU times: total: 31.2 ms
Wall time: 28.3 ms

[13]: 499999500000
```

Рисунок 15. Выполненное задание 3

Взаимодействие с оболочкой системы

Цель: изучить выполнение команд терминала прямо из JupyterLab / Google Colab.

Задания:

1. Выведите список файлов в текущей директории с помощью `!ls`.
2. Проверьте, какой Python используется (`!which python`).
3. Создайте папку `test_folder` (`!mkdir test_folder`) и убедитесь, что она появилась.
4. Переместите файл в новую папку и удалите его.
5. Очистите вывод в ячейке (`!clear`).

Рисунок 16. Задание 4

```
[10]: !dir

Том в устройстве C не имеет метки.
Серийный номер тома: 2CD8-C8FD

Содержимое папки C:\Users\mrkon\lab1ex

23.03.2026 22:54 <DIR>      .
23.03.2026 22:54 <DIR>      ..
23.03.2026 22:24 <DIR>      .ipynb_checkpoints
22.03.2026 23:22      2 035 ex1.ipynb
22.03.2026 23:40     24 175 ex2.ipynb
22.03.2026 23:46      1 270 ex3.ipynb
22.03.2026 23:55     957 ex4.ipynb
22.03.2026 23:56     838 ex5.ipynb
22.03.2026 23:58     872 ex6.ipynb
23.03.2026 00:04    40 139 ex7.ipynb
23.03.2026 02:02      1 856 exerc1.ipynb
23.03.2026 02:14      1 695 exerc2.ipynb
23.03.2026 22:38      8 104 exerc3.ipynb
23.03.2026 22:15     242 script.py
23.03.2026 22:54      1 976 Untitled.ipynb
                12 файлов
                3 папок  7 095 545 856 байт свободно

[12]: !where python

D:\aconda\python.exe
C:\Users\mrkon\AppData\Local\Microsoft\WindowsApps\python.exe

[16]: !mkdir test_folder

[ ]: !rmdir test_folder

[ ]: !clear
```

Рисунок 17. Выполненное задание 4

Работа с Google Drive в Google Colab

Цель: научиться подключать Google Drive, загружать и сохранять файлы, работать с обычными текстовыми файлами.

★ Задания:

1. Подключите Google Drive к Colab с помощью команды:

```
1 from google.colab import drive
2 drive.mount('/content/drive')
```

Проверьте, что ваш диск успешно подключился, используя `!ls /content/drive/MyDrive`.

2. Создайте и сохраните текстовый файл в Google Drive:
 - Откройте файл `my_text_file.txt` в режиме записи.
 - Запишите в него несколько строк текста.
 - Закройте файл и убедитесь, что он появился в папке `MyDrive`.
3. Прочитайте файл из Google Drive:
 - Откройте ранее созданный файл.
 - Считайте его содержимое и выведите в ячейке.

4. Создайте и сохраните CSV-файл вручную, используя Microsoft Excell или Libre Office Calc:
 - Создайте список, в котором будут строки с данными о студентах (например, ФИО, возраст, группа).
 - Запишите этот список в CSV-файл в Google Drive вручную, используя стандартные методы записи в файл.

Рисунок 18. Задание 5

```
from google.colab import drive

drive.mount('/content/drive')

Mounted at /content/drive

file_path = "/content/drive/MyDrive/my_text_file.txt"
with open(file_path, "w") as f:
    f.write("Это тестовый файл, сохранённый в Google Drive.\n")
    f.write("Вторая строка текста")

print("Файл успешно сохранён в Google Drive.")

Файл успешно сохранён в Google Drive.

with open(file_path, "r") as f:
    content = f.read()
    print("Содержание файла:\n", content)

Содержание файла:
Это тестовый файл, сохранённый в Google Drive.
Вторая строка текста

▶ students = [
    ["Червиченко И.Д.", 19, "ИВТ-6-о-24-2"],
    ["Ганиев Д.В.", 19, "ИВТ-6-о-24-2"],
    ["Прохоров В.И.", 20, "ИВТ-6-о-24-2"],
]

csv_path = "/content/drive/MyDrive/students.csv"
with open(csv_path, "w") as f:
    for student in students:
        f.write(",".join(map(str, student)) + "\n")
```

Рисунок 19. Выполненное задание 5

Контрольные вопросы:

1. Основные отличия JupyterLab от Jupyter Notebook?

JupyterLab - это следующее поколение пользовательского интерфейса Project Jupyter. Его главное отличие - интегрированная среда разработки (IDE). В отличие от классического Notebook с одним документом, JupyterLab позволяет открывать несколько ноутбуков, текстовых редакторов, терминалов и консолей в одном окне с вкладками и гибкой компоновкой.

JupyterLab имеет встроенный мощный файловый браузер для навигации, загрузки и управления файлами прямо в интерфейсе.

2. Как создать новую рабочую среду (ноутбук) в JupyterLab?

Нажать кнопку "+" на левой панели инструментов или в меню File -> New Launcher. В открывшейся вкладке Launcher вы увидите раздел "Notebook". Выберите ядро (kernels) для вашего ноутбука (например, Python 3).

3. Какие типы ячеек поддерживаются в JupyterLab и как их переключать?

Основные типы ячеек:

- Code (Код): Для написания и выполнения кода.
 - Markdown: Для форматированного текста, заголовков, списков, формул.
 - Raw (Сырые): Текст, который выводится без преобразований.
- Переключение: Выделите ячейку и используйте выпадающее меню на панели инструментов или горячие клавиши в командном режиме (синяя рамка): у для Code, m для Markdown, r для Raw.

4. Как выполнить код в ячейке и какие горячие клавиши для этого используются?

Выполнить код можно кнопкой "Run" на панели инструментов или горячими клавишами:

- Shift + Enter: Выполнить ячейку и перейти к следующей.
- Ctrl + Enter: Выполнить ячейку и остаться на ней.
- Alt + Enter: Выполнить ячейку и вставить новую снизу.

5. Как запустить терминал или текстовый редактор внутри JupyterLab?

В разделе "Other" Launcher нажмите иконку "Terminal". Текстовый редактор: В разделе "Other" выберите "Text File" для создания нового файла, или просто дважды кликните на существующий текстовый файл (.py, .md, .txt и др.) в файловом браузере.

6. Какие инструменты JupyterLab позволяют работать с файлами и структурами каталогов?

Основной инструмент — файловый браузер (File Browser) на левой боковой панели. Он позволяет создавать, удалять, переименовывать, копировать папки и файлы; загружать файлы с компьютера кнопкой "Upload

Files" (иконка со стрелкой вверх) или перетаскиванием; скачивать файлы на компьютер (правой кнопкой мыши -> "Download"); просматривать структуру проекта.

7. Как можно управлять ядрами (kernels) в JupyterLab?

Управление ядром текущего ноутбука осуществляется через меню Kernel на верхней панели. Там доступны опции:

- Interrupt: Прервать выполнение кода.
- Restart: Перезапустить ядро (очистить все переменные).
- Restart & Run All: Перезапустить ядро и выполнить все ячейки.
- Change Kernel: Сменить ядро на другое (например, с Python 3.8 на Python 3.10 или другое окружение).

8. Каковы основные возможности системы вкладок и окон в интерфейсе JupyterLab?

Вкладки: Каждый открытый документ (ноутбук, терминал, редактор) отображается как вкладка.

Разделение (Split): Вы можете перетаскивать вкладки вправо, влево, вверх или вниз от центра, чтобы создавать новые панели и работать с несколькими документами бок о бок.

Изменение размеров: Границы между панелями можно перетаскивать для изменения размеров окон.

9. Какие магические команды можно использовать в JupyterLab для измерения времени выполнения кода? Приведите примеры.

В JupyterLab (с ядром IPython) используются "магические" команды:

- %time: Измеряет время выполнения одной инструкции.
- %timeit: Запускает код несколько раз и выдает среднее время, лучше подходит для точных замеров небольших фрагментов.
- %%time: Измеряет время выполнения всей ячейки (многострочный код).
- %%timeit: Запускает всю ячейку несколько раз и усредняет результат.

10. Какие магические команды позволяют запускать код на других языках программирования в JupyterLab?

С помощью магии ячеек (Cell Magics) можно запускать код на других языках, если соответствующие ядра установлены:

- `%%bash` (или `%%script bash`): Выполнить ячейку как скрипт Bash.
- `%%html`: Интерпретирует содержимое как HTML.
- `%%javascript` (или `%%js`): Выполнить код JavaScript.
- `%%ruby`: Выполнить код Ruby.
- `%%perl`: Выполнить код Perl.

11. Какие основные отличия Google Colab от JupyterLab?

Colab - это облачный сервис Google, не требующий установки и предоставляющий вычислительные ресурсы (включая GPU/TPU) бесплатно. JupyterLab обычно устанавливается и запускается локально или на сервере, который вы контролируете.

Colab тесно интегрирован с Google Диском. JupyterLab работает с файловой системой сервера, на котором запущен. Colab имеет встроенные функции для совместного редактирования, как в Google Документах.

12. Как создать новый ноутбук в Google Colab?

Перейти на сайт colab.research.google.com. В открывшемся окне нажмите кнопку "Новый ноутбук" (New Notebook) в нижней части диалога. Или через меню: Файл -> Создать -> Новый блокнот.

13. Какие типы ячеек доступны в Google Colab, и как их переключать?

Типы ячеек в Google Colab такие же, как в Jupyter. Переключение: Используйте выпадающее меню в правом верхнем углу ячейки (по умолчанию "Код" или "Текст").

14. Как выполнить код в ячейке Google Colab и какие горячие клавиши для этого используются?

Горячие клавиши полностью аналогичны Jupyter:

- Shift + Enter: Выполнить ячейку и перейти к следующей.
- Ctrl + Enter: Выполнить ячейку и остаться на ней.

- Alt + Enter: Выполнить ячейку и вставить новую снизу.

15. Какие способы загрузки и сохранения файлов поддерживает Google Colab?

- Загрузка с компьютера: Через файловый менеджер (панель слева -> иконка папки -> иконка "Загрузить").
- Сохранение на компьютер: Через файловый менеджер (правая кнопка мыши на файле -> Скачать).
- Монтирование Google Диска: Позволяет читать и сохранять файлы прямо на вашем Диске.
- Загрузка/сохранение по URL: С помощью команд `!wget <URL>` или `!curl -O <URL>` для загрузки, и интеграцией с `google.colab` для сохранения.

16. Как можно подключить Google Drive к Google Colab и работать с файлами?

Выполнить в ячейке код:

```
from google.colab import drive  
drive.mount('/content/drive')
```

После выполнения перейти по ссылке, выбрать аккаунт Google и скопировать код авторизации в поле ввода. После этого ваш Диск будет доступен по пути `/content/drive/MyDrive/`.

17. Какие команды используются для загрузки файлов в Google Colab из локального компьютера?

Самый простой способ - использовать встроенные функции вкладки "Файлы": Откройте панель "Файлы" (иконка папки слева). Нажмите на иконку "Загрузить в хранилище сеанса" (первая иконка с изображением листа бумаги и стрелки вверх). Выберите файл на своем компьютере. Альтернативный способ - код с использованием библиотеки `google.colab`:

```
from google.colab import files  
uploaded = files.upload() # Загруженный файл будет в словаре uploaded
```

18. Как посмотреть список файлов, хранящихся в среде Google Colab?

Использовать левую боковую панель "Файлы" (иконка папки). Там отображается файловая система виртуальной машины. Или выполнить магическую команду оболочки в ячейке: `!ls` (для текущей директории, обычно `/content/`). Чтобы посмотреть все с подробностями: `!ls -la`.

19. Какие магические команды можно использовать в Google Colab для измерения времени выполнения кода? Приведите примеры.

Colab также работает на базе IPython, поэтому все магические команды из JupyterLab работают точно так же: `%time`, `%timeit`, `%%time`.

20. Как можно изменить аппаратные ресурсы в Google Colab (например, переключиться на GPU)?

Откройте меню "Среда выполнения" (Runtime). Выберите пункт "Сменить среду выполнения" (Change runtime type). В открывшемся окне в выпадающем списке "Аппаратный ускоритель" (Hardware accelerator) выберите желаемый вариант: None (нет, только CPU), GPU, TPU. Нажмите "Сохранить". Изменения вступят в силу после перезапуска среды выполнения.

Вывод: были исследованы базовые возможности интерактивных оболочек Jupyter Notebook, JupyterLab и Google Colab для языка программирования Python.